CONTROL-TOWER-LOOP · CLAUDE CODE PLUGIN · F32

The development loop, step by step

A development loop with agents in which the machine dispatches and verifies, and the human decides exactly **three times**: when freezing the spec, when promoting a slice to the queue, and when merging. Everything else — parsing the spec, creating the issues, isolating in worktrees, the claim, the release, detecting residue — the loop does. This page documents every step and the exact format of every artefact that travels between them.

PLUGIN	SLICES CONTRACT	SPEC TEMPLATE	FORKED SKILLS
0.32.0	v16	v2	11 · sp 6.0.3
COMMANDS	HUMAN GATES		
4	3		

---

01 / THE MAP

Three lanes, four phases, three gates

There are **two live sessions per repo with opposite roles**: the coordinator, in the main checkout, which grooms, dispatches, reviews and merges; and one session dispatched per slice, in its own worktree, which implements and stops. The human is not in a lane of execution: the human is at the seams.

HUMAN decides, does not execute · COORDINATOR main checkout · SLICE AGENT .worktrees/<n> · HOOK automatic, no session

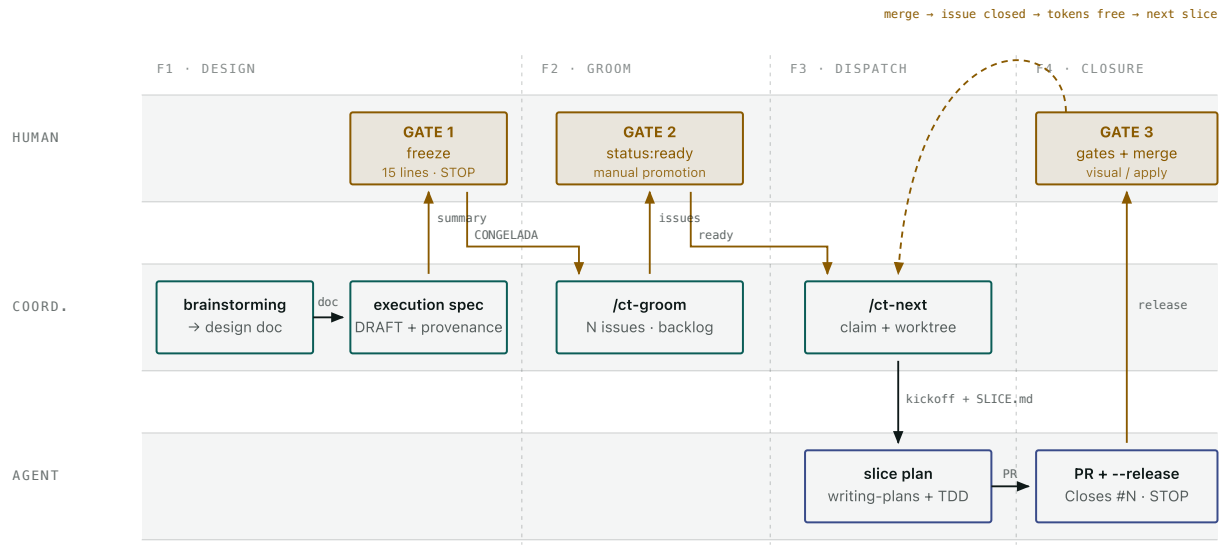

Fig. 1 — The complete cycle of an epic. The amber arrows cross a human seam; the neutral ones are automatic. The dashed arc at the top is the loop's only closure: until a PR is merged and its issue is closed as *completed*, that slice holds on to its area tokens and no neighbour that shares them gets dispatched.

THE TWO-LEVEL MODEL

Control Tower is the subagent-driven development pattern, one level up. Epic level: the slices table is the plan file, `/ct-next` is the coordinator, the dispatched session is the implementer, and the human review of the PR is the *two-stage review*. Slice level: inside the worktree runs the whole set of forked skills — `writing-plans`, `subagent-driven-development`, `test-driven-development`, `systematic-debugging`. The same pattern, two scales.

02 / THE STEPS

Sixteen steps, from the idea to the merge

Every step says who runs it, with which command, what it reads and what it writes. The steps with an amber lane are the ones the machine cannot do.

PHASE 0 **Bootstrap** once per repo · `/ct-init`

S0

● COORD.

Prepare the repo for the loop

The scaffolder seeds the ground and **nothing more**: it is not a planner. It leaves the slices table contract inside `AGENTS.md`, delimited by markers and with its own version, so that whoever writes specs for that repo has the exact header to hand. Filling in the repo's real commands (build, test, lint, CI) in `AGENTS.md` does require exploring.

```
bash ${CLAUDE_PLUGIN_ROOT}/scripts/ct-init.sh "$(pwd)"
```

Two warnings that have to be passed on whole, not resolved: if the repo already came with its own claim protocol, its own worktrees path or its own state file, choosing which one rules is the user's decision. And if it says that *it could not be checked* (no `node`), that is not «there are none».

WRITES	DOES NOT DO
`.agent/STATE.md`, `AGENTS.md` (contract block), `.gitignore` (`.worktrees/` + `.agent/SLICE.md`)	It does not fill in `next_action` — it stays at `(sin slice asignado)`. It does not register the repo in the tower.

PHASE 1 **Design and freeze** epic level · `control-tower-loop:brainstorming`

S1

● COORD.

Explore and ask, one question at a time

Project context first (files, docs, recent commits). Then, **a single question per message**, preferably multiple choice, about purpose, constraints and success criteria. If the assignment describes several independent subsystems, that is pointed out *before* refining details: it gets decomposed into sub-projects, each with its own cycle.

Then 2–3 approaches with their *trade-offs* and one recommendation leading, not an exhaustive list. And the design presented **section by section**, asking for the go-ahead on each one.

THE SKILL'S HARD GATE

Zero code, zero scaffolding, zero implementation skills until the design is approved. In «trivial» projects too.

S2

• COORD.

Write the design doc and self-review it

It is committed to `docs/superpowers/specs/YYYY-MM-DD-<topic>-design.md`. Then four passes with fresh eyes: **placeholders** (TBD, TODO, half-finished sections), **internal consistency**, **scope** (does it fit in one plan?) and **ambiguity** (if something admits two readings, one is chosen and made explicit). It is fixed on the spot; there is no second round.

WRITES	DESTINATION
`…-design.md`, committed	It will be the execution spec's `Handoff origen`. After the freeze it is history: nobody edits it.

S3

• COORD.

Write the execution spec in DRAFT

One file per epic, from `_TEMPLATE-execution-spec.md`. It is not «the join and nothing more»: it is **the record of the join and of its compressed inputs**, with a single criterion for what goes in — *what the executor needs and cannot derive*.

Every frozen decision carries its provenance: `hablada` (with the quote if there is one), `deducida` (it follows from a spoken one or from a rule already written in the repo) or `propuesta` (the agent puts it there). Hard rule: **a `propuesta` is not frozen** — you ask, and then it is spoken, or it drops to «Decisiones aparcadas». The gaps do not get filled in.

WRITES	ALLOWED IN DRAFT
`…-execution.md`, Estado: DRAFT	`[NEEDS CLARIFICATION: …]`
FORBIDDEN HERE	
Each slice's plan. That is written later, against the real code.	

S4

• HUMAN

Gate 1 — the freeze

This gate exists because of one sentence: *«yo no me suelo leer los specs, doy por hecho que lo hablado durante el brainstorming es ok y va al spec»*. If the human does not read the spec, whoever writes it could close decisions with somebody else's signature. The answer is the two pieces of S3 (provenance) and this gate.

At most fifteen lines are presented, in the chat, that fit on one screen: the hypothesis in one or two lines, each frozen decision on one line with its provenance label, and the anti-scope. **And it stops.**

THE OK MUTATES	IF CHANGES ARE ASKED FOR
`DRAFT → CONGELADA` + freeze date	The spec is corrected and the summary is presented again
WITHOUT AN OK	
There is no groom. It is the cycle's only reading, by design.	

PHASE 2 **Groom** spec → GitHub · `/ct-groom`

S5

● COORD.

Dry groom: the freeze gate and the table's validation

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-groom.mjs "<spec>" --repo "<owi
```

Before it even looks at the table, **two checks over the whole spec** — two greps in the pass groom was already making:

- **`[NEEDS CLARIFICATION` unresolved → exit 2.** They are allowed in DRAFT; freezing with a pending gap is invalid. The message names how many there are and the line of the first one.
- **`## Hipótesis` absent or empty → exit 2.** With no falsifiable bet it is not an epic. Work with no bet (maintenance, bugfixes) goes as loose issues. Groom only looks at presence; the *quality* is judged by the human at the freeze.

Then the whole table, and **every failure together in a single exit 2**: a header without `Slice` + `Dep`, the `#` column missing, a gap in the middle of the block, a `#` that is not a pure integer, rows with too many or too few cells, a `Dep` that is unreadable or points at a nonexistent `#` or at itself, a `Gate` outside the vocabulary, or a row that asks for and waives the same gate.

The dry-run validates exactly the same as the real run — a dry-run that validates less is a trap. With `--repo` it is no longer offline: it reads the real issues so that it can compare.

S6

● COORD.

Real groom: one issue per row

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-groom.mjs "<spec>" --repo "<owi
```

Everything it reads comes before everything it writes. Validation, resolution and verification of the link to the spec, enumeration of issues, labels, milestones and — with `--project` — the introspection of the Project v2 (which demands an iteration field named exactly `Sprint` and an iteration that covers today). If it aborts at any of those steps, it has created nothing.

Once validation is passed, the write order is **milestone → labels → issues → registration in the Project with its Sprint**. There is no transaction and it does not pretend there is one: a network failure there leaves what came before created. You get out of a half-way abort **by running it again, not by cleaning up by hand**: the milestone is reused by title, of the labels only the missing ones are created, and the issues are recognised by their `<!-- ct-order:N -->` marker.

SCOPE	IDEMPOTENCE
One invocation = one milestone = one epic. The `#` are 1..N **per epic**: every spec can start at 1.	By existence only. Reporting divergence ≠ converging: without `--reconcile` it never edits an existing issue (exit 3).

S7

● HUMAN

Gate 2 — promote to the queue

Groom creates the issues in `status:backlog`, **never in `status:ready`**. It is a deliberate human gate: `/ct-next` does not consider dispatchable anything that is not in `ready`.

```
gh issue edit <n> --repo <owner/repo> --add-label status:ready --rem
```

If `/ct-next` answers «No hay slices despachables» right after a groom, it is almost certainly this. On finishing, groom reminds you through stderr how many issues of the epic are still in backlog — counting the ones that already existed too, because the criterion is the real state, not «I have just created something».

PHASE 3 Dispatch and implementation

slice level · `/ct-next` + forked skills

S8

● COORD.

Dispatcher dry-run: check what the real run needs

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-next.mjs --repo "<owner/repo>"
```

It verifies that `cmux` is on the PATH, that the agent can be seen from here (a warning only: what really resolves it is the login shell), that the worktree and the

`feat/<n>` branch are free, and that the kickoff renders. **The same checks run in the real run before any claim is written.**

It prints the kickoff as readable prose — the only thing that makes it possible to judge the prompt the agent will receive — and **the whole batch**, with each slice's problem annotated in its own block and a summary of which one would break first.

CHANNEL	ACCOUNT
stdout = the product (plan, selection, reason for blocking). stderr = diagnostics (`aviso:` , `ATENCIÓN:` , aborts).	It always prints which Claude account it has chosen, by which rule, and which wrapper gets typed.

S9

• COORD.

Dispatch: claim, worktree, seed, verified launch

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-next.mjs --repo "<owner/repo>"
```

Five acts, in order, and each one with its reverse if it fails:

- **Diagnostics first.** Order collision, `status:` ambiguity, dependencies outside their section — printed *before* anything is selected. They never block on their own.
- **Selection.** By order within the epic, with the `merge-after` satisfied (issue closed as *completed*), with no collision of `area:` / `touches:` tokens and with room under `--cap` .
- **Claim in code,** not by prompt: `dispatch-check <issue> --repo <repo>` moves `ready → in-progress` BEFORE creating the worktree.
- **Isolation.** `.worktrees/<n>` + branch `feat/<n>` , seed of `.agent/SLICE.md` , and the ignore rule written into the common directory's `info/exclude` — **with verification of the effect**: if `git status` still sees the file, the dispatch is aborted and the claim is reverted.
- **Launch with a sentinel.** `cmux --command` does not execute: it *types*. A short line is typed that sources a launcher on disk (the kickoff travels by file, where no prompt can bite it), and that launcher writes a sentinel with the real `$PWD` . With no sentinel nothing is called «launched»: the typing is retried every 2500 ms until the 15 s budget runs out.

THE RULE THAT REMAINS

If it cannot be confirmed that the agent started, the result cannot be «launched» with exit 0. And with the sentinel absent when the cap runs out **the claim is not reverted either**: reverting it with an agent that starts late would destroy live work, whereas the residue declared with its exact commands is recoverable.

S10

● HOOK

Hydration and verification-first start-up

The `SessionStart` hook injects the state into every new session of the worktree. The precedence rule is the same for every reader: **if `.agent/SLICE.md` exists, that is the state; if not, `.agent/STATE.md`**. The file's presence is the signal for «I am in a slice worktree» — there is no environment variable and no path detection. And every message names the file it really read, not a constant.

The agent's first act is not to touch code: **confirm `pwd`/branch, `git log`, and a green baseline before anything else.**

IF BLOCKED IS SET

The hook announces it prominently and declares the `next_action` SUSPENDED. Lifting it is an explicit human decision.

STOP HOOK

It blocks the end of the turn if there is work above `last_commit` left unrecorded. A commit that only touches the state file does not count.

S11

● AGENT

First act: the slice plan, against the real code

The epic's plan (`## Enfoque técnico`) comes before the table. The **slice plan** comes after, and it is written by the dispatched agent on starting up with `control-tower-loop:writing-plans` **using the issue as the spec**: its ACs in EARS, its `## Out of scope / Protected` and its `## Contexto del epic` are exactly the input writing-plans asks for.

It is saved in `docs/superpowers/plans/` and committed: **it travels in the PR**. With the plan written, the «is there a plan?» diamond of `subagent-driven-development` finds a plan and the skill re-engages whole.

WHY HERE

It is written against the real code, at the right moment. A plan written in the spec would be a plan written blind.

S12

AGENT

Implement: a subagent per task, TDD, two reviews

`control-tower-loop:subagent-driven-development` with TDD: a fresh subagent per task, review between tasks, fast iteration. The issue's `Acepta` are **1:1 with tests** — that is what the `## Acceptance criteria (EARS, 1:1 con tests)` heading means.

What the agent does **not** do, said in the kickoff and not only in the skills: it does not merge the PR, it does not start the next slice, and **it does not create new worktrees** — the dispatcher already put the isolation in place.

IF IT GETS BLOCKED

`blocked: {reason, unblock}` in `.agent/SLICE.md`, **never as prose inside `next_action`**. The dispatcher reads that field and reports it with the reason the agent itself wrote.

S13

AGENT

Close the slice: PR, release, stop

`finishing-a-development-branch` has a new step 0: if `.agent/SLICE.md` exists, **there is no menu**. The path is fixed: green tests → push + PR → literal release command → **STOP**.

```
node <dispatch-check> <issue> --repo <owner/repo> --release
```

The PR carries `Closes #N` **in the body** — not in the title, not in a comment: GitHub only interprets the closing keywords in the PR body and in commit messages. And it is not cosmetic: it is the only thing that closes the issue on merging. A merged PR with its issue open leaves the slice holding on to its tokens for ever, blocks the serialising lane, and no dependent ever sees its `merge-after` satisfied.

--RELEASE FREES

The `--cap`: it stops counting as a live agent.

--RELEASE DOES NOT FREE

The `area:` / `touches:` tokens. An `in-review` holds them **until the merge**.

CONTAMINATION GATE

It refuses with exit 5 if the branch introduces `.agent/STATE.md` or `.agent/SLICE.md`, if the base cannot be determined, or if the resolved base is the same commit as HEAD.

PHASE 4 **Closure and harvest** human + `/ct-status`

S14

● HUMAN

Gate 3 — review, close gates, decide

Two human gates, closed vocabulary: `visual` — a human has to SEE the change, with a screenshot or video of the before/after in the PR; `apply` — nothing is applied against a real environment until a human reviews the plan or the dry-run.

The loop **writes and shows** the gates — `gate:<token>` label, `## Gates` section of the issue, an explicit line in the kickoff, `gates` field of the `SLICE.md` — but **it does not stop you merging with one left open**. The one who closes them is the human.

```
node ${CLAUDE_PLUGIN_ROOT}/scripts/ct-status.mjs --repo "<owner/repo
```

`/ct-status` answers in one go the question the coordinator used to answer by hand: what is in flight, what has been delivered and what is residue. **It mutates nothing** — not one write along the whole path, with a test that checks it by looking at the real argv `gh` was called with. And no read carries `--limit`: everything paginates.

> A RULE THE REPORT CANNOT APPLY FOR YOU
>
> Whatever you check before merging has to be able to stop the merge. This command prints; it blocks nothing. A check whose result does not stop the next action is decoration, not a check.

S15

● HUMAN

Merge — the only act that unblocks the neighbours

The merge fires the whole chain: `Closes #N` closes the issue as *completed* → the `area:` / `touches:` tokens are freed → every dependent's `merge-after` becomes satisfied. Neither `--release`, nor `--reopen`, nor anything automatic does this.

PRACTICAL CONSEQUENCE	KNOWN TRAP
An unmerged PR holds up its area neighbours. Reopening does not unblock them either: the only thing that unblocks them is the merge.	The closing keywords of **any commit message** that reaches the default branch apply, and quotes do not protect. The dispatcher *warns* when a dependency is recorded as closed by a loose commit that belongs to no merged PR — but it does not block.

S16

● COORD.

Harvest the residue and record the evidence

Every path in the plugin that deletes a worktree is a failure path; **none covers success**. Observed in the field: PR merged, issue closed, and `.worktrees/451` + `feat/451` still on disk with their agent alive for thirteen hours. With six slices that is six checkouts, six dead branches and six zombie agents — and a finished slice's worktree **blocks the redispatch of that number**.

```
git worktree remove .worktrees/<n>   &&   git branch -d feat/<n>
```

Every run of `/ct-next` crosses the merged issues against the disk and emits `cosecha pendiente:` with the exact commands. **It deletes nothing**, and that is deliberate: «merged» is not «nobody is touching that», and deleting a worktree is irreversible. Lastly, the spec's `## Registro de cierre` collects the evidence each gate was closed with — a gate closed without evidence is a gate not closed.

AND BACK TO S8

With the tokens free and the worktree harvested, the epic's next slice is dispatchable.

The edges back

Three exits from the happy path, all deliberately manual: nothing automatic invokes them.

SITUATION	COMMAND	TRANSITION	WHAT IT CHECKS
PR rejected in review — it is fixed on top of what is there	`dispatch-check <n> --reopen`	`in-review → **in-progress**`	It demands exactly `status:in-review` (exit 2 if not). It goes `in-progress` and to `ready` because `ready` holds no to and a neighbour w be dispatched ove base that does no contain that work. deletes nothing fr the disk: it prints w is left.
Abandon the slice — start from scratch, or break a dead claim	`dispatch-check <n> --requeue`	`in-progress → **ready**`	The only transition frees tokens witho merge, so it check before declaring: i demands exactly `in-progress` and neither `.worktrees/<n>` `feat/<n>` is left. I could not look, it refuses: what has been seen is not declared absent.
Work blocked — it cannot go on	The `blocked` field of the `SLICE.md`	`— (no transition)`	No transition of th loop gets it out of there: stale-claim detection does no it (worktree and br exist), `--requeue` refuses for that ve reason, and `--release` would lying. The dispatch reads it and warns **There is no auton fix: it is a human decision.**

03 / THE STATE

A slice is a label

There is no database. A slice's state is the labels of its GitHub issue, and the issue's timeline records every transition with its timestamp. That makes the state survive a `/clear`, a redispatch and another machine — and it makes two different resources, the `--cap` and the area tokens, be freed at different moments.

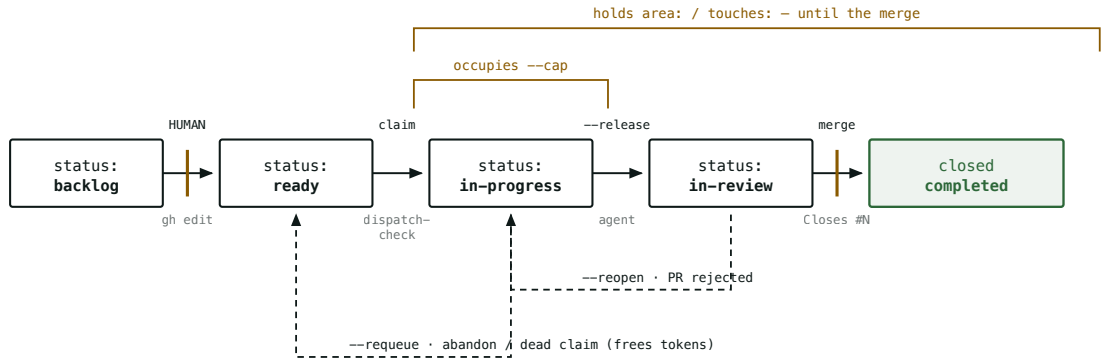

Fig. 2 — The five states and their two edges back. The vertical amber bars mark the two transitions only a human runs. Note the asymmetry that explains most of the real blockages: `--release` frees the `--cap` but **not** the area tokens, so an open unmerged PR holds up its neighbours even when no agent is running.

	STATUS:IN-PROGRESS	STATUS:IN-REVIEW
Occupies `--cap`	yes (there is an agent running)	**no**
Holds `area:` / `touches:`	yes	**yes, until the merge**
Satisfies a `merge-after`	no	no (it has to be closed as *completed*)
Stale-claim detection	yes (crossed against worktree / branch / cmux session)	no — having no session there is normal

04 / ARTEFACTS

Eleven formats, and what travels in each one

The principle that orders the whole catalogue: **what you write outside the slices table and outside `## Contexto del epic` does not reach the agent**. The agent that implements a slice does not receive the spec — it receives a start-up prompt and the body of the issue. A demand written in another section is invisible however forcefully it is worded.

`docs/superpowers/specs/YYYY-MM-DD-<topic>-design.md`

● COORDINATOR

The design validated in the brainstorming, approved section by section. It has no rigid template: it covers architecture, components, data flow, error handling and testing, with each section scaled to its complexity — a few sentences if it is straightforward, up to 200–300 words if it has nuance.

WRITES	READS	LIFE
`brainstorming` , step 6	The human, section by section, during the design	Committed. After the freeze it is **history**: nobody edits it
DESTINATION		
It is the execution spec's `Handoff origen`		

`docs/superpowers/specs/YYYY-MM-DD-<topic>-execution.md`

● COORDINATOR

The central artefact of the epic level. One file per epic, tracked, `DRAFT → CONGELADA`, archived when the epic closes. Seven sections plus a header, and of all of them **only one travels to the agent**.

TEMPLATE	PARSED BY	GATE
`_TEMPLATE-execution-spec.md` v2	`/ct-groom` : the table, `## Contexto del epic` , `## Hipótesis` and the `[NEEDS CLARIFICATION` markers	Without `Estado: CONGELADA` there is no groom

FORMAT

```
# <Epic> — Execution spec

**Handoff origen:** `docs/superpowers/specs/YYYY-MM-DD-<topic>-design.md`
**Fecha de congelación:** [the human's OK sets it, not the session that writes]
**Estado:** DRAFT

## Hipótesis del experimento       ← REQUIRED. groom: exit 2 if it is missing or
**The bet:** [what we believe will happen — worded so that it can be FALSE]
**How we will know it failed:** [the concrete observation that knocks it down]
**Anti-scope — what this epic does NOT do:** [the only thing that stops it growi

## Decisiones congeladas              ← one per line, with its Procedencia
- **D-1 · <short title>** — <the decision>. *(Procedencia: hablada — «literal qu
- **D-2 · <short title>** — <the decision>. *(Procedencia: deducida de D-1.)*

## Enfoque técnico                    ← the EPIC's plan. Área/Toca/Dep come out
## Contexto del epic                  ← travels to the agent (together with ## D
The invariants go INSIDE: version floors, copy rules, naming,
paths that are not touched. One line each, with exact values.

## Tabla de slices                    ← contract v16, see below. Only ONE per sp
## Decisiones aparcadas (BLOCKED)     ← gaps that were never spoken. Empty at th
| ID | Fila | Qué falta decidir | Opciones vistas | Estado |

## Registro de cierre (evidencia)     ← filled in as each slice closes
| Slice | specReviewedSha | codeReviewedSha | uiScreenshot | Gate cerrado con |
```

§ Tabla de slices — the contract with /ct-groom

● PARSED

It is the only part of a spec that a program parses. The contract lives in the repo's `AGENTS.md` (`ct-init:slices-contract` block, versioned, maintained by `/ct-init`) and is not duplicated in the spec. The table is located **by its column header** `Slice` + `Dep`, never by a section number.

EXACT HEADER, COPYABLE AS IT STANDS

```
| # | Slice | Tipo | Entrega | Dep | Acepta | Protegido | Área | Toca | Gate |
|---|-------|------|---------|-----|--------|-----------|------|------|------|
| 1 | walking skeleton | ui | empty navigable screen | — | the /hoy route answer
| 2 | scoring          | backend | POST /score endpoint — consumes getPillars()
```

COLUMN	REQUIRED	WHERE IT LANDS
`#`	yes	A pure integer (`1`, `2`…). The slice's order and the target of `Dep`. Never `S1` nor `**1**`: the whole row is discarded. Unique **within its milestone**, not within the repo.
`Slice`	yes	**The issue's title** (`#N <Slice>`). It is also the first line of the kickoff and the name of the cmux workspace — which is why it must be short and readable, not a sentence.
`Tipo`	no	`type:<valor>` label + which **technical addendum** the agent receives. Recognised: `ui`, `backend`, `infra`, `bugfix`. Another value does not abort, it warns.
`Entrega`	no	The issue's `## Descripción` section. It **no longer** feeds the title.
`Dep`	—	`#N` of the **order** of this same table, separated by commas. It feeds the `merge-after` graph. In the issue it goes between backticks on purpose: a bare `#N` would be auto-linked by GitHub to issue N.
`Acepta`	no	The `## Acceptance criteria (EARS, 1:1 con tests)` section, one per line. **The comma always separates** — escape it with `\,`.
`Protegido`	no	The `## Out of scope / Protected` section. Free text in one piece: it is not split by commas.
`Área` / `Toca`	no	`area:<x>` / `touches:<y>` labels: the tokens the dispatcher detects collisions with and forces serialisation by. Without these columns, that machinery is inert for that slice.
`Gate`	no	**Closed** vocabulary: `visual`, `apply`. An unknown value **aborts** (unlike `Tipo`). Waive it with a leading `!`. An empty cell = «I have declared nothing», **not** «I waive everything».

«No value» is written `—`, `–`, `−`, `-`, `--` or as an empty cell: every dash variant counts, because a phone with autocorrect swaps one for another without you noticing.

The 4 cell rules

They go in the template and not in the contract because **a human judges them at the freeze, not a parser**.

- **1 · Clarified = convergence.** A row is ready when two independent readings converge on the same outcome. If you can imagine it two ways, it is not: `[NEEDS CLARIFICATION: …]` and go and ask.

- **2 · `Acepta` = postconditions, not actions.** Every criterion asserts the *observable resulting state* («an expired token leaves the session at login», not «the token is refreshed»). EARS disambiguates; the 1:1 test denotes it. The incompleteness of the criteria is almost all under-specified result, not forgotten action.
- **3 · `Gate` = the residue of the `Acepta`.** Whatever is observable and *cannot* have a 1:1 test goes to the Gate — that is where the human comes in. The default by `Tipo` is a heuristic, not the rule.
- **4 · `Dep` declares an interface.** *The most important of the four.* The `Entrega` of a row with a `Dep` names **what it consumes from the previous slice** — the function, the endpoint, the token, the file. A `Dep` that only says `#2` is an undeclared overlap, and the undeclared overlap is what gets reviews rejected.

`The body of the GitHub issue — what the agent really reads` `● GENERATED`

The order of the sections is not cosmetic: **what it delivers → with what context it is interpreted → how it is verified → what is missing before it can be merged → what stays out**. A known heading is recognised by **exact equality**, not by prefix: a `## Acceptance criteria propuestos por QA (borrador)` written by a human is never confused with the real section.

FORMAT, IN ORDER

```
> Slice `#1` del epic. Spec: [docs/…/spec-execution.md § Tabla de slices](https:

## Descripción                          ← only if `Entrega` carries a real val
<text of Entrega>

## Contexto del epic                    ← only if the spec carries text. IDENT
<copied verbatim from the spec; --reconcile keeps it up to date>

## Contexto heredado                    ← ALWAYS, with a placeholder. groom NE
<the coordinator writes it when something already merged conditions THIS slice>

## Acceptance criteria (EARS, 1:1 con tests)
- <criterion 1>
- <criterion 2>

## Dependencias                         ← only if there are deps
*(cada `#N` de esta sección es el ORDEN del slice, NO un número de issue)*
- merge-after `#1`

## Gates                                ← ALWAYS, also when there is none
<resolved gates, or the assertion that there are none>

## Out of scope / Protected             ← ALWAYS
- 🚫 <Protegido>      |  - (ninguno declarado)

<!-- ct-order:1 -->                        ← greppable order marker. Bookkeepi
```

YOUR ZONE, AND WHERE IT ENDS

Everything **between `## Contexto heredado` and `## Acceptance criteria` is yours**: `--reconcile` does not write a single byte there, neither in the headings you paste nor in the prose you write around them. It survives you pasting the body of another whole issue, which is exactly what is needed («slice #2 left this set up»).

The end of your zone is marked by `## Acceptance criteria`. If that heading **does not appear exactly once**, there is no way of knowing where yours ends — and then `--reconcile` writes nothing beyond the inherited context, gives up out loud and asks you to restore it. Of the two possible errors, the expensive one is the one that deletes hand-written text.

What `/ct-groom` compares in an issue that already exists, and what it does not

Label vocabulary

GENERATED

The labels are not decorative metadata: they are the executable state of the loop. They are created **only if they do not already exist** in the repo, and groom names through stderr the new ones alongside the reused ones — which is how you detect that a spec has invented a synonym (`area:hoy` against `area:today`) instead of reusing the vocabulary, which is the only thing that makes collision detection work.

PREFIX	VALUES	WHO MOVES IT	WHAT IT DECIDES
`status:`	`backlog` · `ready` · `in-progress` · `in-review` · `blocked`	groom creates in `backlog`; the human promotes; `dispatch-check` claims and frees	Dispatchability, `--cap`, token retention
`type:`	the `Tipo` column — recognised `ui`, `backend`, `infra`, `bugfix`	groom	The kickoff's technical addendum + the default gate
`area:`	the `Área` column	groom	Collision token: two slices with the same area do not run at the same time
`touches:`	the `Toca` column	groom	The same, plus the **global serialising lane**: `migration`, `ci`, `pbxproj`
`gate:`	`visual` · `apply` · `none`	groom (from `Gate`, or derived from `Tipo`)	The human gate that survives a redispatch, a `--reopen` and a `/clear`

> `gate:none` exists because **silence cannot mean two different things**: without that label, «this slice demands no gate» and «this issue predates the gates» would be indistinguishable. An issue groomed before they existed falls back to `Tipo`, so it does not lose the gate it already had.

The kickoff — the start-up prompt of the dispatched agent

GENERATED

It is not a file in the repo: it is rendered on every dispatch and written into a temporary launcher that cmux sources. Eleven lines, and the order matters — **what is enumerated gets read, and what is left to «it will work it out» competes with the rest of the issue body**. The arbitrary-prose sections are *named* instead of interpolated, because this whole thing gets typed into a pty.

STRUCTURE

```
1. Estás implementando UN slice (<Slice>) del repo <owner/repo>, issue #N (orden
2. Es human-gated: NO mergees el PR, NO empieces el siguiente slice, y NO crees
   worktrees nuevos — ya estás en el que te preparó el dispatcher.
3. Arranque verification-first: confirma pwd/rama, git log, y baseline verde
   ANTES de tocar nada.
4. Hidrátate de .agent/SLICE.md y del issue; los criterios de aceptación son
   <AC interpolados>. Lee además "## Out of scope / Protected": eso NO se toca.
5. Lee también "## Contexto del epic" y "## Contexto heredado". Si alguna está
   vacía o no aparece, no hay nada que heredar — no lo busques fuera del issue.
6. Primer acto, con el baseline verde: escribe el plan del slice con
   control-tower-loop:writing-plans usando el issue como spec. Guárdalo en
   docs/superpowers/plans/ y commitéalo: viaja en el PR.
7. Con el plan commiteado y el gate 'plan' con OK humano, conduce la implementac
   consultando ct-step (next → report → controls → juez ct-judge → verdict → com
   comitea ct-step, nunca tú ni el implementador) hasta "run delivered".
8. <addendum técnico según Tipo: ui | backend | infra | bugfix>
9. <líneas de GATE — qué tiene que ver o revisar un humano antes del merge>
10. Si el trabajo queda BLOQUEADO, márcalo en .agent/SLICE.md como
   `blocked: {reason, unblock}` — NO en prosa dentro de next_action.
11. Al acabar: commit refs al issue, actualiza .agent/SLICE.md, abre el PR contr
   <base> con `Closes #N` en el CUERPO del PR (no en el título, no en un
   comentario), libera el claim con `node <dispatch-check> N --repo <r> --releas
   deja el estado mergeable y PARA.
   + el POR QUÉ de ese Closes, aparte: es lo ÚNICO que cierra el issue al mergea
```

`.worktrees/<n>/.agent/SLICE.md` ● SLICE AGENT

The slice's state. **Ignored by git, never committable** — and the guarantee is not entrusted to the documentation: `/ct-next` writes the rule into the common directory's `info/exclude` (one write covers the coordinator and all the worktrees) and **then verifies the effect**. The presence of this file is the signal for «I am in a slice worktree».

SEEDED BY	WRITTEN BY	READ BY
`/ct-next` on dispatch	The slice agent, turn after turn	The `SessionStart` and `Stop` hooks, and `/ct-next` (to detect blockages)
GIT **ignored** · `--release` exits with exit 5 if the branch introduces it		

THE SEED, EXACTLY AS IT IS WRITTEN

```
---
task: "<Slice>"
role: "slice-agent (sesión DESPACHADA por /ct-next): implementas ESTE slice y PA
  No groomeas, no mergeas, no despachas el siguiente."
gates: "visual — GATES HUMANOS pendientes: los cierra quien revisa el PR, NO tú.
       # or: "ninguno (este slice no exige ningún gate humano antes de mergear)"
status: not_started
branch: "feat/<n>"
base: main
last_commit: "<base SHA>"         # a SHA, not a branch name: without it the Sto
github_issue: <n>
you_are_here: "worktree fresco para el issue #N de GitHub (orden #M)"
next_action: "hidrátate del issue y empieza por el primer AC (<AC-1>)"
blocked: null                     # explicit, not omitted
verify: ""                        # the PENDING check, never a fact already chec
tasks: []
---
## Current State
(slice recién despachado, sin trabajo aún)
```

> `role` and `gates` are **fields** and not sentences inside a prompt for the same reason as `blocked`: what has to survive a re-hydration is a field. The kickoff is lost with its session's context; this file is injected by the hook into every new session of the worktree. No code in the plugin decides anything with `role` or with `gates` — the executable gate is the labels.

`.agent/STATE.md` (checkout principal) ● COORDINATOR

The other state file, and the one that talks about **no** slice. It is tracked and the dispatcher does not touch it: in a slice's worktree it stays at zero diff. The same fields as the `SLICE.md`,

with an opposite `role`.

```
role: "coordinador (checkout principal): groomeas, despachas con /ct-next, revis
  mergeas. NO implementas slices aquí — eso pasa en .worktrees/<n>."
next_action: "(sin slice asignado)"      # /ct-init leaves it like this on purpo
blocked: null
```

> WHY `/CT-INIT` DOES NOT FILL IN `NEXT_ACTION`
>
> It is a scaffolder, not a planner. If it points at some pending item found around the repo, the `SessionStart` hook will hydrate **all** the future sessions of that repo believing that is what comes next — even when it has nothing to do with what is about to be done.

docs/superpowers/plans/YYYY-MM-DD-<feature>.md ● SLICE AGENT

The slice plan, written against the real code and committed in the PR. It is written assuming an engineer **with zero context of the codebase**: exact paths always, complete code at every step, exact commands with their expected output.

REQUIRED HEADER + THE STRUCTURE OF ONE TASK

```
# <Feature> Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: use control-tower-loop:subagent-d
> development (recommended) or control-tower-loop:executing-plans. Steps use `-

**Goal:** <one sentence>
**Architecture:** <2-3 sentences>
**Tech Stack:** <key technologies>

## Global Constraints
[the spec's invariants — version floors, dependency limits, naming
and copy rules — one line each, with the exact values copied verbatim.
The requirements of EVERY task include them implicitly.]

---

### Task N: <Component>

**Files:**
- Create: `exact/path/to/file.ts`
- Modify: `exact/path/to/existing.ts:123-145`
- Test:   `tests/exact/path/to/test.ts`

**Interfaces:**
- Consumes: [what it uses from earlier tasks — exact signatures]
- Produces: [what the later ones rely on — names, parameter and return types.
  A task's implementer sees only their own: this block is how they learn the
  names their neighbours use.]

- [ ] **Step 1: Write the failing test**      ← with the test's code, not a desc
- [ ] **Step 2: Run test to verify it fails** ← with the command and the expecte
- [ ] **Step 3: Write minimal implementation**
- [ ] **Step 4: Run test to verify it passes**
- [ ] **Step 5: Commit**
```

PLAN FAILURES — NEVER WRITTEN

«TBD», «implement later», «add appropriate error handling», «handle edge cases», «write tests for the above» without the code, «similar to Task N» (the code is repeated: the engineer may read the tasks in another order), and references to types or functions that no task defines.

The PR ● AGENT ● HUMAN

The slice's delivery package. It contains the code, the tests, **the committed plan** and — if the slice carries a `visual` gate — the before/after screenshot. It is opened against the dispatch's base and the agent **stops**.

BRANCH	REQUIRED IN THE BODY	WHO MERGES
`feat/<n>` — deterministic, it is what makes it possible to cross it with the issue	`Closes #N` — not in the title, not in a comment	The human. Always.

`.agent/conventions-ack.md` · HUMAN

The acknowledgement that makes a correct warning **stop warning without deleting correct documentation**. Before it existed, the only way of silencing a warning was to delete the text that fired it.

```
claim: 2026-07-28 — manda el claim del plugin; scripts/dispatch-check.sh se queda
  para trabajo a mano fuera del loop
residuo-status: 2026-07-28 — #101, #102 <por qué se quedan así>
```

It silences **that** signal and only that one; the rest keep warning, and on the following runs a one-line note remains saying that it is silenced and since when — «decided not to look at this» and «there is nothing here» are not the same. All three things are needed (signal, date and reason): a line that does not parse is reported and silences nothing. And an acknowledgement **with no numbers** silences nothing: there is no wildcard for «all», which would give back the same static silence the mechanism corrects.

05 / SIGNALS

The exit codes, which are the real interface

The four executables share a channel convention — **stdout is the product, stderr is the diagnostics** — and a grammar of three states: done, could not be checked, something is left pending. If `/ct-next` runs inside a `/loop`, this table is what decides whether to carry on, stop or retry.

`/ct-groom`

EXIT	MEANS	WHAT TO DO
0	No real divergence, or `--reconcile` resolved it entirely	Nothing
1	The run stopped dead: one of the epic's two scope gates, a `gh` read failure, a write failure half-way, or something about `--project` that could not be taken as good	Read the message: it says `no se ha creado ni modificado nada` when that is so
2	The table is invalid, **or the spec does not pass the freeze gate** (`[NEEDS CLARIFICATION` pending, `## Hipótesis` absent or empty)	Fix the spec. Every failure comes out together, not one at a time
3	Something real is left unreconciled: divergence of title/link/labels/AC/deps, a duplicated section, or an orphan issue	Review it. **Watch out with `groom --dry-run && groom`**: the dry-run's `3` cuts the chain exactly when there is something to apply

`/ct-next`

EXIT	MEANS	WHAT TO DO
`0`	Progress (something was launched, in whole or in part), or there was nothing selectable **and it has already explained why**	Carry on as normal
`1`	Something broke or was left half-done that a human has to resolve: an orphan issue in `in-progress`, an unmet precondition, or at least one slice **launched without verification**	Stop. Faced with an unverified launch, **look at the cmux session BEFORE deleting anything**: reverting the claim with a live agent is worse than leaving it
`2`	A usage or static-configuration error: badly placed flags, a binary name that is not a bare command	Correct the invocation
`3`	A batch was selected and it ended with zero launches **without anything being left half-done**: every candidate was skipped at claim time. No claim written, no branch and no worktree created	Retry later. It is not an alarm signal, and now the message can say so truthfully
`130 / 143`	SIGINT / SIGTERM. An interruption is **always acknowledged**, even when it arrives with the batch already finished	If it arrived between the claim and the worktree, the claim has already been reverted automatically

`/ct-status` and `dispatch-check`

COMMAND	EXIT	MEANS
`ct-status`	`0`	Nothing to review: nothing in flight without signs of life, no residue, no half-done read
	`3`	There is something to review: residue, a claim with no live process, orphan labels, deliveries not harvested
	`1`	**It could not be checked.** The precedence is `1` > `3` > `0`, never the other way round: an incomplete read is not a loop at rest, and a `0` over truncated data is exactly the failure this command came to eliminate
`dispatch-check`	`1`	A collision detected in time, or a race lost with a clean revert. **A normal result of the protocol**: that slice is skipped and the batch carries on
	`3`	A one-off infrastructure failure — treated the same as a collision, but the message makes it explicit that it is a `gh` that failed
	`4`	**Orphan issue**: a race lost or a readback failure, and the revert afterwards failed too. The issue is left stuck in `in-progress` with nobody working on it. This **stops the whole batch**
	`5`	`--release` refuses: the branch introduces a state file, the base cannot be determined, or the resolved base is the same commit as HEAD

06 / LIMITS

What the loop does not guarantee

Written down here because a limit that is known and said is operable, and an implicit one is not.

THE CLAIM IS NOT ATOMIC

The lock `dispatch-check` claims an issue with is GitHub labels, **with no compare-and-swap**. It is reproduced and verified —not suspected— that two dispatchers launched almost at the same time can claim the same shared token and both start. There is no wait and no retry that closes this gap today.

The mitigation is operational: do not launch two `/ct-next` at the same time against the same repo. The plan is to migrate the claim to a real atomic primitive via git refs; until then, this is the only guarantee that exists.

NOTHING WATCHES THE CLAIMS BETWEEN INVOCATIONS

The claim is a label, with no heartbeat: the only one who finds out is whoever is running `/ct-next` or `/ct-status` at that moment. And the evidence of life is **local to this machine**, so «abandoned» is never asserted, only «there is no trace of it here». If the loop is dispatched from more than one place, «no sign of life» means «nobody is working on it *here*».

- **The gates are written and shown; they are not enforced.** The loop does not stop you merging with a gate left open. The human does.
- **There is no transaction in the groom.** Once validation is passed, a network failure leaves what came before created. There is no rollback and it does not pretend there is one: you get out of a half-way abort by running it again.
- **The link to the spec points at the default branch.** If the file is moved or renamed, the link of the issues already created dies. It is detected on the next run, but it is not fixed without `--reconcile`. **Groom after pushing the spec.**
- **A `merge-after #N` written outside its section gates nothing** — it warns, but it dispatches all the same. The domain of detection and the domain of application are, deliberately, the same: only the recognised section.
- **`dist/` is what runs, not `hooks/`.** Every change to the hooks has to carry a rebuilt `dist/` in the same commit, or the repo distributes the old hook while the source says something else. And the suite stays green, because `npm test` builds first: it tests a freshly made `dist/` while the committed one rots.
- **The measure is pre-registered, not harvested.** The GitHub timeline already records every label transition with its timestamp — half the instrumentation exists uncollected. The death

> criterion still stands: *if the loop costs more human intervention than it saves, it is said and it stops.*

Composed from the code and the documentation of the `control-tower-loop` plugin at `/Users/jpereag/Documents/control-tower-plugin`: the four `commands/*.md`, the forked skills of `skills/`, the generators of `scripts/groom.js` and `scripts/kickoff.js`, the v16 contract of `scripts/ct-init.sh`, and the v2 template of the execution spec. Every format on this page comes from the source that emits it, not from a second-hand description.